

Mini Project Report
on
ML-Powered Medical Diagnosis Assistant

Submitted by

Team: Rachith Bharadwaj T N and team

Include the team members present for the lab session below

Rachith Bharadwaj T N 24BDS062

Dhanush Gowda N 24BDS018

Shreedhar M Kadkol 24BDS076

Kishan Kumar Y 24BDS031

Under the guidance of

Dr. Utkarsh Mahadeo Khaire

Department of DSAI



DEPARTMENT OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DHARWAD

11/11/2025

Certificate

This is to certify that the project, entitled **ML-Powered Medical Diagnosis Assistant**, is a bonafide record of the Mini Project coursework presented by the students whose names are given below during **Academic Year 2024-25** in partial fulfilment of the requirements of the degree of Bachelor of Technology in Data Science and Artificial Intelligence.

Roll No	Names of Students
---------	-------------------

24BDS062	Rachith Bharadwaj T N
----------	-----------------------

24BDS018	Dhanush Gowda N
----------	-----------------

24BDS076	Shreedhar M Kadkol
----------	--------------------

24BDS031	Kishan Kumar Y
----------	----------------

Dr. Utkarsh Mahadeo Khaire
(Project Supervisor)

Contents

List of Figures	4
List of Tables	5
1 Introduction	6
1.1 Background	6
1.2 Motivation	6
1.3 Objectives	6
2 Problem Statement	6
2.1 Healthcare Challenges in Resource-Limited Areas	6
2.1.1 Limited Access to Medical Professionals	6
2.1.2 High Rate of Preliminary Misclassification	7
2.1.3 Connectivity and Infrastructure Limitations	7
2.1.4 Cost and Time Barriers	7
2.2 Impact Assessment	7
3 Proposed Solution	7
3.1 System Overview	7
3.1.1 Machine Learning Diagnosis Module	7
3.1.2 Aggregate Evidence Validation	7
3.1.3 Village-Point Offline Dashboard	7
3.1.4 Public Demonstration Link	8
3.1.5 Data Management and Security	8
3.2 System Architecture	8
4 Technical Implementation	9
4.1 Technology Stack	9
4.2 Machine Learning Models	9
5 Data and Methods	9
5.1 Dataset Description	9
5.2 Data Preprocessing	9
6 Results and Discussions	10
6.1 Model Performance	10
6.2 Model Comparison and Dataset Distribution	10
6.3 System Interface	10
6.4 Diagnostic Report Sample	11
7 System Features and Capabilities	12
8 Development and Testing	12
8.1 Development Phases	12
8.2 Testing Strategy	12
8.3 Testing Results	13
9 Future Enhancements	13

9.1	Planned Features	13
9.2	Scalability Plans	13
10	Conclusion	13
	References	14

List of Figures

1	System Architecture Diagram	8
2	Model Comparison Chart	10
3	Disease Distribution Chart	11

List of Tables

1	Technology Stack Overview	9
2	Model Performance Metrics	10
3	Sample Diagnostic Output	12

1 Introduction

1.1 Background

Healthcare accessibility remains one of the most important challenges in rural and resource-limited areas. Many patients in village settings do not receive timely preliminary screening because doctors, laboratories, internet access, and transport facilities may not be available at the same time. A simple local diagnostic support tool can help health volunteers collect symptoms, run a first-level analysis, and guide the patient toward the correct next action.

The ML-Powered Medical Diagnosis Assistant is developed as an educational prototype for preliminary symptom-based triage. The current implementation runs as a Flask dashboard with a trained TensorFlow model, local evidence support, fracture screening, urgency labels, and encrypted local logging. The dashboard can run on the same laptop, on devices connected to the same hotspot or local network, and through a public ngrok link for demonstration.

1.2 Motivation

The project is motivated by the need for a practical village-point healthcare support system:

1. Bridge the healthcare gap by providing quick preliminary diagnosis support near the patient.
2. Reduce confusion in symptom reporting by converting free-text symptom input into structured model features.
3. Support offline typed symptom diagnosis when internet is unavailable.
4. Help health workers prioritize urgent cases and recommend professional consultation.
5. Preserve patient privacy through local encrypted event logs.

1.3 Objectives

1. Develop a symptom-based disease prediction model for educational triage.
2. Build a runnable Flask Web-App dashboard for text input, browser voice input, ranked predictions, and printable reports.
3. Provide village-point access through laptop, hotspot, or local network operation.
4. Publish a demonstration link through ngrok for presentation and review.
5. Add aggregate reference evidence using cleaned public clinical-style datasets.
6. Include COVID screening, fracture screening, urgency labels, and recommended actions.
7. Validate the system with saved metrics and 100+ clinical scenario test cases.

2 Problem Statement

2.1 Healthcare Challenges in Resource-Limited Areas

2.1.1 Limited Access to Medical Professionals

Rural areas often face a shortage of doctors and specialists. Patients may need to travel long distances for even basic consultation, which delays care and increases the chance of complications.

2.1.2 High Rate of Preliminary Misclassification

Symptoms may be described differently by different patients. Language barriers, unclear symptom vocabulary, and lack of diagnostic equipment can make early triage difficult for field workers.

2.1.3 Connectivity and Infrastructure Limitations

Many village clinics and temporary health camps cannot depend on stable internet. Therefore, the system must support local operation without relying on online front-end assets. Public internet access is useful for demonstrations, but the field workflow must remain usable locally.

2.1.4 Cost and Time Barriers

Travel, waiting time, and repeated visits create financial pressure for patients. A local symptom-screening dashboard can help identify urgent cases earlier and reduce unnecessary delays.

2.2 Impact Assessment

The system does not replace doctors, but it can support preliminary triage by organizing symptoms, showing likely disease classes, and highlighting urgency. This is especially useful in school projects, health-awareness camps, and village point demonstrations.

3 Proposed Solution

3.1 System Overview

The proposed system combines a local machine learning model, a Flask dashboard, aggregate reference evidence, fracture screening, and encrypted local logging.

3.1.1 Machine Learning Diagnosis Module

The main diagnosis path uses a TensorFlow neural-network model trained from the cleaned symptom-disease dataset. The saved model predicts across 42 disease classes using 134 symptom features. The dashboard returns a primary diagnosis, confidence score, matched symptoms, and alternative ranked predictions.

3.1.2 Aggregate Evidence Validation

The implementation uses a Spark-backed aggregate validation index built from cleaned public clinical-style CSV datasets. This is not live hospital EHR access. It stores only aggregate disease and symptom counts, which are displayed as reference evidence with the diagnosis.

3.1.3 Village-Point Offline Dashboard

The dashboard is designed to run on a local laptop for village-point use. Typed symptom input, model inference, local datasets, CSS, JavaScript, and encrypted logs are all local after installation. Browser voice input is included when supported by the browser, but typed symptom entry is the reliable offline workflow.

3.1.4 Public Demonstration Link

For presentation, the local dashboard is published through ngrok at:

`https://demetra-varietal-cindi.ngrok-free.dev`

The public link requires internet, while the village-point workflow can run locally or on the same hotspot or LAN.

3.1.5 Data Management and Security

Diagnosis and feedback events are stored locally. When the required cryptography package is available, logs are written as AES-256-GCM encrypted JSON lines. Local logs and instance keys are excluded from Git tracking.

3.2 System Architecture

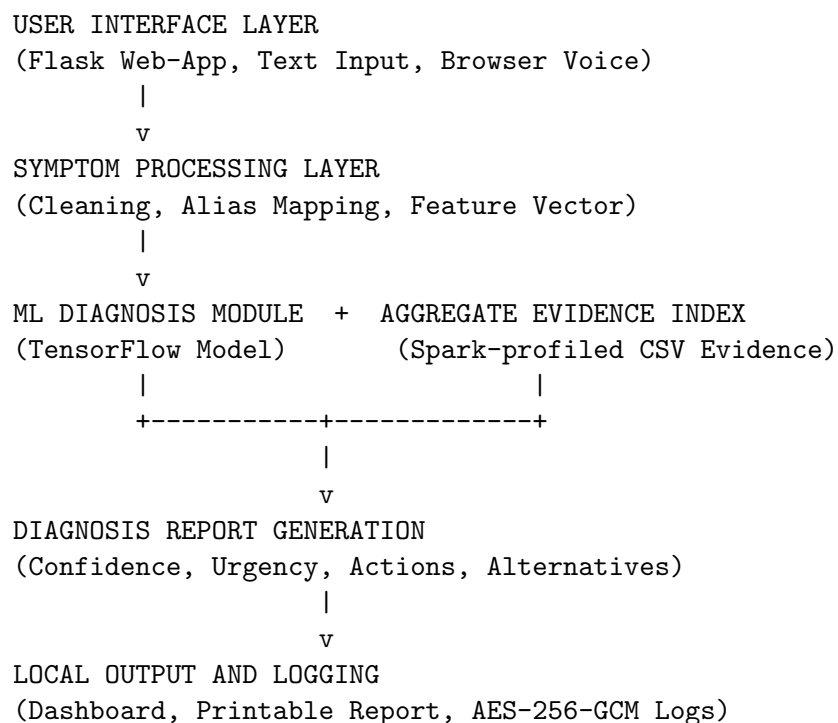


Figure 1: System Architecture Diagram

4 Technical Implementation

4.1 Technology Stack

Table 1: Technology Stack Overview

Layer	Technology	Purpose
Backend	Python 3.x, Flask	Web routes, API endpoints, core logic
ML/AI	TensorFlow, scikit-learn, NumPy	Model inference, training, evaluation
Big Data	Apache Spark, Pandas	Aggregate validation index and dataset profiling
Frontend	HTML5, CSS3, JavaScript	Dashboard, symptom input, result display
Deployment	Local laptop, hotspot/LAN, ngrok	Village-point and public demo access
Datasets	Cleaned symptom and clinical-style CSV files	Training, testing, reference evidence
Security	AES-256-GCM, local file storage	Encrypted diagnosis and feedback logs

4.2 Machine Learning Models

The saved runnable model is a TensorFlow neural network. The training workflow also compares Logistic Regression, Random Forest, Gradient Boosting, and Support Vector Machine baselines. The trained model artifact is stored as:

BACKEND/models/ml_model.h5

5 Data and Methods

5.1 Dataset Description

The system uses cleaned public CSV datasets stored in the **DATASETS** folder. The main disease classifier is based on symptom columns mapped to disease labels. Additional cleaned datasets support COVID examples, fracture screening, patient profiles, and aggregate reference evidence.

- **Training and Testing Files:** Symptom features and disease labels for model building and saved-model evaluation.
- **Fracture Dataset:** Injury symptoms mapped to 11 fracture screening types.
- **Clinical-Style Reference Files:** Public cleaned CSV sources used for aggregate validation evidence.

5.2 Data Preprocessing

Preprocessing includes label normalization, binary symptom feature conversion, duplicate-row removal, vocabulary building, and symptom alias mapping. The web app also maps common patient

phrases such as “shortness of breath” or “loss of smell” to the corresponding model symptom vocabulary.

6 Results and Discussions

6.1 Model Performance

The saved model was evaluated on the included clean testing file. Current metrics are shown in Table 2.

Table 2: Model Performance Metrics

Metric	Value
Accuracy	0.9762
Precision	0.9643
Recall	0.9762
F1-Score	0.9683
Accuracy 95 percent CI	0.8768 to 0.9958
Manual Clinical Scenario Tests	100+ completed

These results should be interpreted as classroom dataset performance, not as proof of real-world clinical accuracy.

6.2 Model Comparison and Dataset Distribution

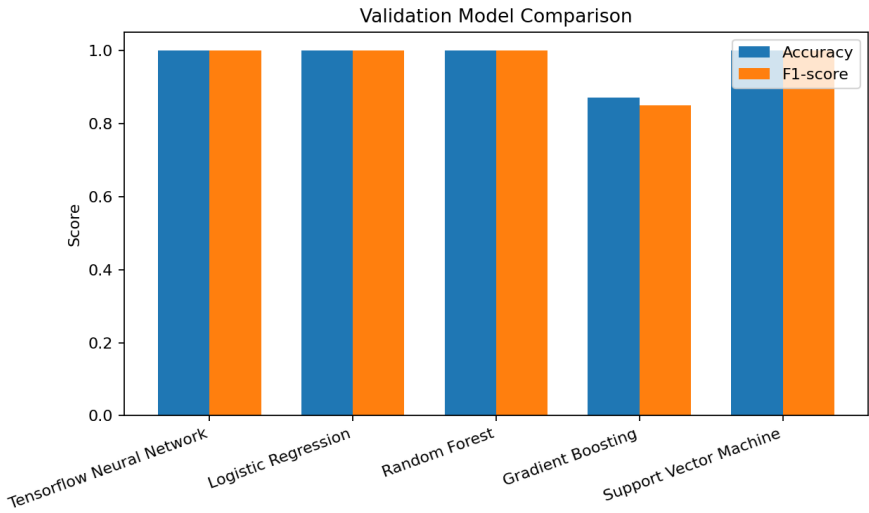


Figure 2: Model Comparison Chart

6.3 System Interface

The dashboard provides symptom input, quick example chips, voice input when the browser supports it, model readiness status, evidence status, and printable diagnosis results. It can be opened on the host laptop through:

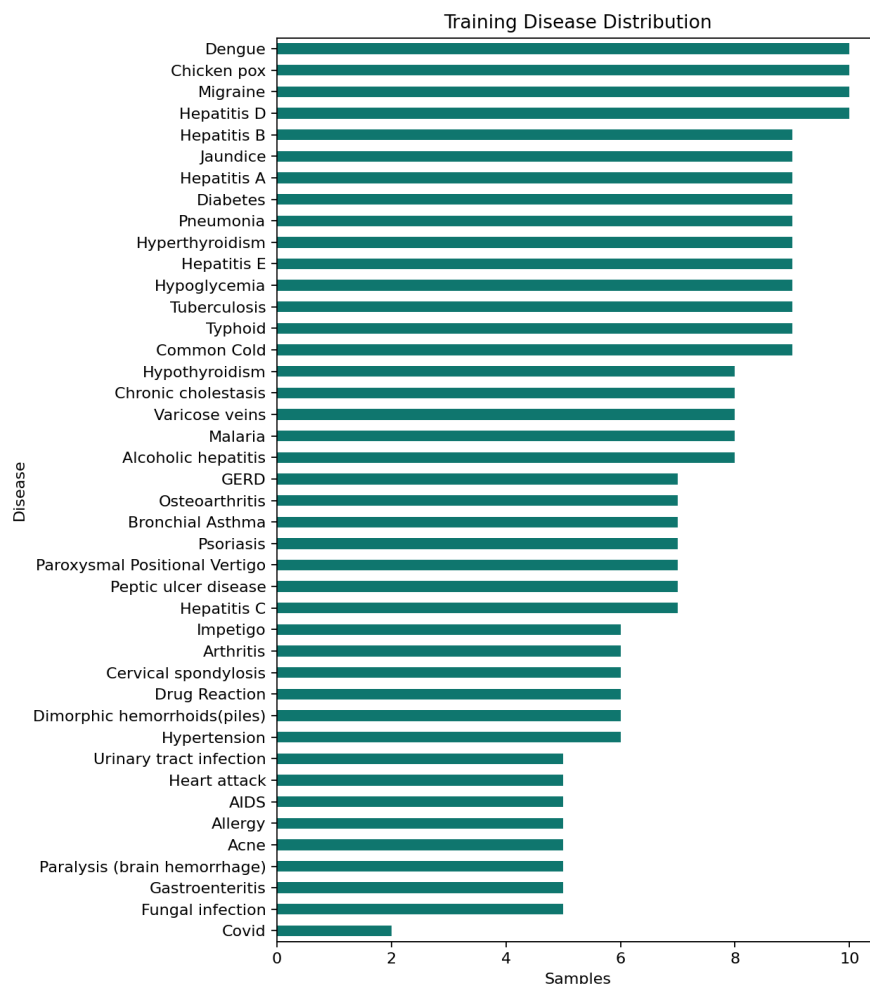


Figure 3: Disease Distribution Chart

`http://127.0.0.1:5000`

Devices on the same hotspot or local network can open the dashboard using the host laptop IP address:

`http://laptop-ip-address:5000`

For public demonstration, the site link is:

`https://demetra-varietal-cindi.ngrok-free.dev`

6.4 Diagnostic Report Sample

A typical result includes the primary diagnosis, confidence percentage, urgency level, matched symptoms, reference cases, recommended action, and alternative diagnoses. Example verified public demo output:

Table 3: Sample Diagnostic Output

Input Symptoms	High fever, cough, tiredness, loss of taste and loss of smell
Primary Diagnosis	Covid
Confidence	72 percent
Urgency Level	Moderate
Matched Symptoms	5

7 System Features and Capabilities

The current system supports:

- 42 disease classes and 134 symptom features.
- 11 fracture screening types.
- COVID hallmark symptom screening.
- Ranked predictions with confidence scores.
- Matched symptom evidence and aggregate reference-case counts.
- Urgency level classification and suggested action text.
- Printable diagnosis reports.
- AES-256-GCM encrypted local diagnosis and feedback logs.
- Public demo through ngrok and local village-point operation.

8 Development and Testing

8.1 Development Phases

1. Data cleaning and preprocessing.
2. Model training and baseline comparison.
3. Saved-model evaluation and metric artifact generation.
4. Flask dashboard implementation.
5. Aggregate validation index generation.
6. COVID and fracture screening paths.
7. Encrypted local logging.
8. ngrok public demo and village-point local access setup.

8.2 Testing Strategy

Testing included unit-level checks for model loading, API health checks, diagnosis requests, local network access checks, ngrok public endpoint checks, and manual clinical scenario validation.

8.3 Testing Results

- Health endpoint verified model readiness, evidence index readiness, fracture index readiness, and encrypted logging.
- Public ngrok endpoint verified at <https://demetra-varietal-cindi.ngrok-free.dev>.
- Same-network dashboard access verified through the laptop IP address.
- Public diagnosis test returned a COVID prediction for hallmark COVID symptoms.
- Local fracture test returned a tibial/fibular fracture prediction for fall injury symptoms.
- 100+ comprehensive clinical scenario test cases were completed.
- Frontend dependency check confirmed no online CDN requirement for typed diagnosis workflow.

9 Future Enhancements

9.1 Planned Features

1. Native Android packaging for field-device distribution.
2. Fully offline native speech-to-text using tools such as Vosk or CMU Sphinx.
3. Multi-language symptom entry for Indian languages.
4. Doctor review workflow for confirmed cases.
5. Improved calibration of confidence scores.

9.2 Scalability Plans

The system can be expanded through edge deployment for rural clinics, privacy-safe model updates, additional curated datasets, and controlled cloud deployment for urban demonstrations.

10 Conclusion

The ML-Powered Medical Diagnosis Assistant demonstrates a practical educational prototype for preliminary symptom-based diagnosis. The current implementation is stronger than a concept-only system because it runs as a real Flask dashboard, loads a saved TensorFlow model, shows evidence and urgency, supports local encrypted logs, and can be used through a public ngrok link or as a local village-point dashboard.

The system is not a medical device and must not replace qualified healthcare advice. However, it can support triage education, health-camp demonstrations, and early symptom organization in resource-limited settings. The verified public site link is:

<https://demetra-varietal-cindi.ngrok-free.dev>

References

- [1] Pallets Projects. Flask Web Development Documentation. Retrieved from <https://flask.palletsprojects.com/>
- [2] Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., and Zheng, X. (2016). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. Retrieved from <https://www.tensorflow.org/>
- [3] Apache Software Foundation. Apache Spark Documentation. Retrieved from <https://spark.apache.org/documentation.html>
- [4] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., and others. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [5] World Health Organization. Universal Health Coverage. Retrieved from <https://www.who.int/health-topics/universal-health-coverage>
- [6] Daemen, J., and Rijmen, V. (2002). *The Design of Rijndael: AES-The Advanced Encryption Standard*. Springer-Verlag.
- [7] Topol, E. J. (2019). High-Performance Medicine: The Convergence of Human and Artificial Intelligence. *Nature Medicine*, 25(1), 44-56.
- [8] Rajkomar, A., Dean, J., and Kohane, I. (2019). Machine Learning in Medicine. *New England Journal of Medicine*, 380(14), 1347-1358.
- [9] ngrok. Secure tunnels to localhost. Retrieved from <https://ngrok.com/>
- [10] Project Public Demo. Retrieved from <https://demetra-varietal-cindi.ngrok-free.dev>